

## Spring framework

---

### Q. what is spring?

---

Spring is light weight and popular open source java framework design by rod Johnson in 2003 and it is known framework of framework because using spring framework we can develop and design any kind of application means console base application, web base application, enterprise level application etc.

Suppose consider before spring we have struts it is only for develop web base application, hibernate help us to develop only database application means they have limitation to work with specific type of application but spring overcome this problem and built framework for develop any kind of application or framework is flexible for develop any kind of application so its name is spring.

### What is Spring Framework?

Spring is a lightweight and popular open-source Java-based framework developed by Rod Johnson in 2003. It is used to develop enterprise-level applications. It provides support to many other frameworks such as Hibernate, Tapestry, EJB, JSF, Struts, etc., so it is also called a framework of frameworks. It's an application framework and IOC (Inversion of Control) container for the Java platform. The spring contains several modules like IOC, AOP, DAO, Context, WEB MVC, etc.

### Why to use Spring?

Spring framework is a Java platform that is open source. Rod Johnson created it, and it was first released under the Apache 2.0 license in June 2003.

When it comes to size and transparency, spring is a featherweight. Spring framework's basic version is about 2MB in size.

The Spring Framework's core capabilities can be used to create any Java program, however there are modifications for constructing web applications on top of the Java EE platform. By offering a POJO-based programming model, the Spring framework aims to make J2EE development easier to use and to promote good programming habits.

### Prerequisite knowledge to learn spring

---

1. Core JAVA 2. JDBC 3. Servlet 4. JSP 5. SQL and DBMS Concept etc

### Q. Why use spring framework?

---

1. Provide loose coupling feature
2. Provide dependency injection and inversion of control facility
3. To develop the secure web application with the help of spring security
4. Provide easy support to database communication with the help of DAO and ORM module
5. Provide MVC pattern for develop web application
6. Easy for testing

7. Support integration with other frameworks like as hibernate etc
  8. Able to develop any kind of application like as console to enterprise level application
- Etc.

**Spring framework provide different types of sub module to us to work with different kind of application**

- 1. Spring core:** this module help us to design the console base application and provide implementation of all core java concept means this is the root module of spring without knowledge of this module developer cannot work with another module.
- 2. Spring DAO/JDBC:** DAO stands data access object means this module helps us to connect your spring application with database or work with database by using JDBC Concept.
- 3. Spring ORM:** ORM stands for object relationship mapping and this module help us to connect your spring application with database by using ORM concept like as hibernate or JPA.
- 4. Spring MVC:** MVC stands for module view and controller so it is module of spring framework help us to design web application by using MVC concept.
- 5. Spring Security:** this module helps us to design web application with security layers.
- 6. Spring AOP:** AOP stands for aspect oriented programming. It can be defined as the breaking of code into different modules, also known as modularisation where the aspect is the key unit of modularity.

If we want to work with spring framework we need to know dependency injection concept

### Q. what is dependency injection?

---

Dependency injection means if particular method or constructor is dependent on specified parameter it may be primitive type of data or object type of data called as dependency injection

Means we cannot call that method without parameter called as dependency injection

Spring framework injects that parameter internally by XML configuration or by using annotation configuration called as dependency injection in spring framework.

```
class Employee
{
    private int id;
    private String name;
    public void setId(int id)
    { this.id=id; }
    public int getId(){
        return id;
    }
    public void setName(String name)
    { this.name=name; }
    public String getName(){
        return name;
    }
}
```

```
Employee emp = new Employee();

emp.setId(1);
emp.setName("ABC")
```

If we think about above code we have method name as setId() and setName() these method required

Parameter if we want to call it means we cannot call these method cannot call without parameter from function calling point means these method dependent on particular type of parameter so it is called as dependency injection

**Note:** above code indicate the setter injection

## Types of dependency injection

---

**1. Setter injection:** if we define the setter and getter method then we required to pass parameter in setter method means we cannot setter method without parameter called as setter injection

This is the simpler of the two DI methods. In this, the DI will be injected with the help of setter and/or getter methods. Now to set the DI as SDI in the bean, it is done through the **bean-configuration file** For this, the property to be set with the SDI is declared under the **<property>** tag in the bean-config file.

### Following code demonstrate the setter injection

---

```
class Employee
{ private int id;
  private String name;
  private int sal;

  public void setId(int id) //work with setter injection
  { this.id=id;
  }
  public int getId(){
    return id;
  }
  public void setName(String name)
  { this.name=name;
  }
  public String getName(){
    return name;
  }
  public void setSal(int sal)
  { this.sal=sal;
  }
  public int getSal(){
    return sal;
  }
}
```

**2. Constructor injection:** If we pass parameter to constructor called as constructor injection

In this, the DI will be injected with the help of [constructors](#). Now to set the DI as CDI in bean, it is done through the **bean-configuration** file. For this, the property to be set with the CDI is declared under the **<constructor-arg>** tag in the bean-config file.

```
class Employee
{ private int id;
  private String name;
  private int sal;
  public Employee(String name, int id, int sal)
  { this.name=name;
    this.id=id;
    this.sal=sal;
  }
  public void setId(int id)
  { this.id=id;
  }
  public int getId(){
    return id;
  }
  public void setName(String name)
  { this.name=name;
  }
  public String getName(){
    return name;
  }
  public void setSal(int sal)
  { this.sal=sal;
  }
  public int getSal(){
    return sal;
  }
}
```

**3. Collection dependency:** if we pass collection as parameter in setter function or in constructor called as collection dependency with setter injection or constructor injection

```
import java.util.*;
class Company
{ public void setNames(List<String> empNames)
  {
  }
}
```

**4. Map Dependency:** when we pass map as parameter in function called as map dependency.

```
import java.util.*;
```

```

class Company
{ public void setNames(Map<Integer, String> empNames)
{
}
}

```

Or

```

Classs Company{
    Public Company(Map<Integer,String> emp){
    }
}

```

**5. Property dependency:** if we pass property as parameter in function or constructor called as property dependency

```

import java.util.*;
class Company
{ public void setNames(Properties<Integer, String> empNames)
{
}
}

```

**6. Object dependency:** object dependency means if we pass object of one class as parameter in another function or create object of one class in another class called as object dependency.

```

import java.util.*;
class Employee{
}
class Company
{ Employee employee;
    public void setEmployee(Employee employee)
    { this.employee=employee;
    }
}

```

**Note:** if we think about above code we have two classes one is Employee class and one is Company class and we pass employee class object as parameter in setEmployee() function means we cannot call setEmployee() function without employee class object means we can say setEmployee() function is dependent on Employee object so it is object dependency and if we want to real time scenario above code say company class is dependent on employee means company is dependent on employee without employee any company cannot execute.

**Differentiate between constructor injection and setter injection.**

Constructor Injection vs. Setter Injection

| Constructor Injection                                      | Setter Injection                                             |
|------------------------------------------------------------|--------------------------------------------------------------|
| There is no partial injection.                             | There can be partial injection.                              |
| It doesn't override the setter property.                   | It overrides the constructor property.                       |
| It will create a new instance if any modification is done. | It will not create new instance if any modification is done. |
| It works better for many properties.                       | It works better for few properties.                          |

| Spring IoC (Inversion of Control)                                                                                                                           | Spring Dependency Injection                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Spring IoC Container is the core of Spring Framework. It creates the objects, configures and assembles their dependencies, manages their entire life cycle. | Spring Dependency injection is a way to inject the dependency of a framework component by the following ways of spring: Constructor Injection and Setter Injection |
| Spring helps in creating objects, managing objects, configurations, etc. because of IoC (Inversion of Control).                                             | Spring framework helps in the creation of loosely-coupled applications because of Dependency Injection.                                                            |
| Spring IoC is achieved through Dependency Injection.                                                                                                        | Dependency Injection is the method of providing the dependencies and Inversion of Control is the end result of Dependency Injection.                               |
| IoC is a design principle where the control flow of the program is inverted.                                                                                | Dependency Injection is one of the subtypes of the IOC principle.                                                                                                  |
| <a href="#">Aspect-Oriented Programming</a> is one way to implement Inversion of Control.                                                                   | In case of any changes in business requirements, no code change is required.                                                                                       |

Now we want to discuss about the spring core module

## How to create application by using spring core

### Steps to develop the application by using spring core module

#### 1. Create Maven project using eclipse

##### Q. what is maven?

Maven is POM based tool POM stands project object model and this tool help us to download the required libraries from internet in java application or required .jar from internet using java application Means programmers not download manually .jar from google and not need to add it manually in class path of application

**Maven** is a powerful project management tool that is based on POM (project object model). It is used for project build, dependency, and documentation. It simplifies the build process like ANT(Another Neat

**Tool**). But it is too much more advanced than ANT. In short terms we can tell maven is a tool that can be used for building and managing any Java-based project. Maven makes the day-to-day work of Java developers easier and generally helps with the comprehension of any Java-based project.

**Note:** when we use maven then your system must be connect to internet.

### Q. why use maven?

- 
1. download required jar file from internet add in project class path
  2. When we shift the project location from one computer to computer then we not need to change class path of .jar maven can download jar from internet add in class path of project

Maven provide pom.xml file to us and in this file just we need to add dependency or required dependency so maven automatically download .jar as per the dependency

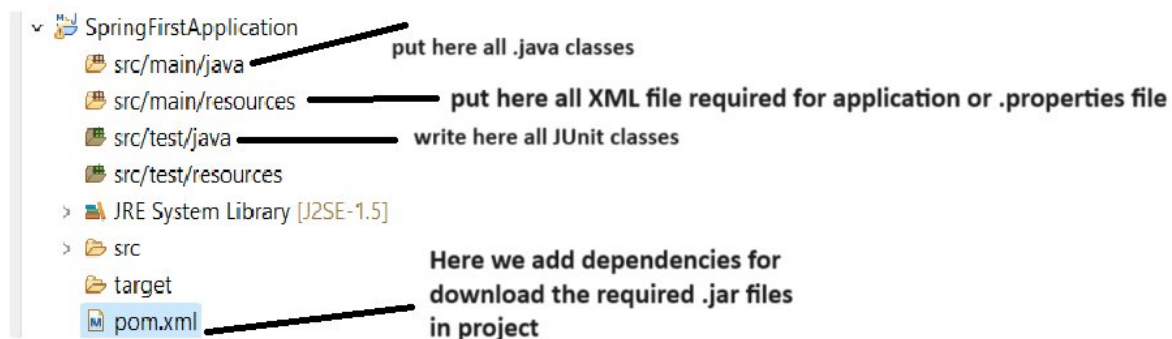
### How to create maven project using eclipse

#### Steps

---

open eclipse ---- file --- new -- others --- select maven option ---- select maven project ----click on next button ---- select check box(create simple project and skip archetype selection) ---- click on next button - --- give group id(package name) and artifact id( project name) -----click on finish button

Once we create maven project we get following folder structure or project structure given below



### 2. Add following maven dependencies

---

Dependency is basically XML tag in pom.xml file which help us to download the required .jar file for application

**If we want to add dependency in pom.xml file we have to add following tags in pom.xml**

<Dependencies>

Write here dependency tags  
</dependencies>

**IF we want to work with spring core module we have to following dependencies**

**a. spring core dependency**

**b. spring context**

```
<dependencies>
<!-- https://mvnrepository.com/artifact/org.springframework/spring-core -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-core</artifactId>
  <version>5.3.17</version>
</dependency>
<!-- https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>5.3.17</version>
</dependency>

</dependencies>
```

Once we add dependencies in spring application we can create POJO class

### 3. Create POJO/bean class

---

Here we want to create Employee as POJO class.

```
package org.techhub;

public class Employee {
    private int id;
    private String name;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }
}
```



```

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getSal() {
        return sal;
    }

    public void setSal(int sal) {
        this.sal = sal;
    }
    private int sal;
}

```

#### 4. Configure bean class by using XML or by using annotation

---

In the case of spring every POJO class consider as bean class so for perform dependency injection and inversion of control we need configure bean class by using XML or using annotation

##### How to configure bean using XML

---

If we want to configure bean using XML we have following steps.

##### a) Create xml file under src/main/resources folder

right click on src/main/resources folder --- select new option ---- other --- xml ---- xml file -- click on next button --- give xml file name --- click on next and finish

##### b) Write doc type in XML file

---

#### Q. What is doctype in XML?

---

XML is custom tag language and it used for

1. Maintain user defined database
2. To share data using REST API
3. To write Custom configuration files

means when we create your tag and if we want to reuse that multiple you can create doc type for that tag and using doc type you can reuse tag again and again not need to write every time

Here spring team design own spring configuration tag and spring users i.e java developer want to reuse that tag for configure spring application for that purpose java developer need to write doctype provided by spring team in his XML file for writing spring bean configuration using XML

## How to write doctype in Spring XML File

---

Doctype always present in .jar of framework which we downloaded by maven repository so in the case of spring core if we want to write doc type for configure bean using XML we have following steps.

### Steps to find doc type and use in XML

---

click on maven dependencies ---- select spring beans.jar ---- select package org.springframework.beans.factory.xml --- open spring-beans.dtd file ---- the copy doc type from commented section file shown below

**Open xml file which we create in src/main/resources folder and paste this doctype on top in file**

```
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">
```

Once we write doc type in XML file you can write bean configuration tag in XML file.

### c) Write bean configuration tag in XML

---

```
<beans>
  <bean id="ref" class="packagename.classname">
    <property name="fieldname" value=""/>
  </bean>
</beans>
```

**<beans>**: this tag is used for write more than one bean configuration in it.

**<bean id="ref" class="packagename.classname">**:

Here bean indicate the class and id indicate reference of bean class which is created by spring container and class indicate bean class name

**<property>** indicate setter method which we write in POJO class and name indicate variable name with setter method and value indicate parameter pass to setter method by spring container.

### Example shown in following code

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">

<beans>
```

```
<bean id="e" class="org.techhub.Employee">
  <property name="id" value="1"/>
  <property name="name" value="ram"/>
  <property name="sal" value="10000"/>
</bean>
</beans>
```

**Once we configure XML file you can perform dependency injection and inversion of control**

## 5. Create Client Application

---

Client application means class with main method and which is used for load the XML file where we configure bean class and create object of bean class as well as perform dependency injection

### Steps to Create client application

---

#### 1. Load XML file from src/main/resource folder

If we want load XML file from class path we have following syntax

```
Resource r = new ClassPathResource("filename.xml");
```

**Description:** Here resource is interface and ClassPathResource is implementer class of Resource interface and which is load the XML file from classpath

#### Sample code

---

```
package org.techhub;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
public class ClientApplication {
    public static void main(String[] args) {
        Resource r=new ClassPathResource("test.xml");
    }
}
```

## 2. Create Spring Container

---

### Q. what is spring container?

---

Spring container is internal API which is used for manage the overall activity of spring framework like as dependency injection, inversion of control, manage bean scope, manage life cycle of bean etc.

The Spring container is the core of the Spring Framework. The container will create the objects, wire them together, configure them, and manage their complete life cycle from creation till destruction. The Spring container uses DI to manage the components that make up an application

## There are two types of Spring Container

### 1. BeanFactory

The first and foremost thing when we talk about Spring is dependency injection which is possible because Spring is a container and behaves as a factory of Beans. Just like the BeanFactory interface is the simplest container providing an advanced configuration mechanism to instantiate, configure, and manage the life cycle of beans.

Beans are Java objects that are configured at run-time by Spring IoC Container. BeanFactory represents a basic IoC container which is a parent interface of ApplicationContext. BeanFactory uses Beans and their dependencies metadata to create and configure them at run-time. BeanFactory loads the bean definitions and dependency amongst the beans based on a configuration file (XML) or the beans can be directly returned when required using Java Configuration. There are other types of configuration files like LDAP, RDMS, properties files, etc. BeanFactory does not support Annotation-based configuration whereas ApplicationContext does.

### 2. ApplicationContext

ApplicationContext belongs to the Spring framework. Spring IoC container is responsible for instantiating, wiring, configuring, and managing the entire life cycle of beans or objects. BeanFactory and ApplicationContext represent the Spring IoC Containers. ApplicationContext is the sub-interface of BeanFactory. It is used when we are creating an enterprise-level application or web application. ApplicationContext is the superset of BeanFactory, whatever features provided by BeanFactory are also provided by ApplicationContext.

Now we want to use BeanFactory interface

If we want to use BeanFactory we have use following syntax

**Syntax:** BeanFactory ref = new XMLBeanFactory(Resource)

#### **Description:**

Here XMLBeanFactory is implementer class of BeanFactory and which accept Resource reference means load the bean classes for creating its object at run time and manage the dependency injection

#### **Q. what is difference between BeanFactory and ApplicationContext?**

**Note:** we will discuss this question later in this chapter

| BeanFactory                                                                                                | ApplicationContext                                                                                                                     |
|------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| It is a fundamental container that provides the basic functionality for managing beans.                    | It is an advanced container that extends the BeanFactory that provides all basic functionality and adds some advanced features.        |
| It is suitable to build standalone applications.                                                           | It is suitable to build Web applications, integration with AOP modules, ORM and distributed applications.                              |
| It supports only Singleton and Prototype bean scopes.                                                      | It supports all types of bean scopes such as Singleton, Prototype, Request, Session etc.                                               |
| It does not support Annotations. In Bean Autowiring, we need to configure the properties in XML file only. | It supports Annotation based configuration in Bean Autowiring.                                                                         |
| This interface does not provides messaging (i18n or internationalization) functionality.                   | ApplicationContext interface extends MessageSource interface, thus it provides messaging (i18n or internationalization) functionality. |
| BeanFactory does not support Event publication functionality.                                              | Event handling in the ApplicationContext is provided through the ApplicationEvent class and ApplicationListener interface.             |

|                                                                                                                                                                                                                |                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BeanFactory does not support Event publication functionality.                                                                                                                                                  | Event handling in the ApplicationContext is provided through the ApplicationEvent class and ApplicationListener interface.                                             |
| In BeanFactory, we need to manually register BeanPostProcessors and BeanFactoryPostProcessors.                                                                                                                 | The ApplicationContext automatically registers BeanFactoryPostProcessor and BeanPostProcessor at startup.                                                              |
| BeanFactory will create a bean object when the getBean() method is called thus making it Lazy initialization.                                                                                                  | ApplicationContext loads all the beans and creates objects at the time of startup only thus making it Eager initialization.                                            |
| BeanFactory interface provides basic features only thus requires less memory. For standalone applications where the basic features are enough and when memory consumption is critical, we can use BeanFactory. | ApplicationContext provides all the basic features and advanced features, including several that are geared towards enterprise applications thus requires more memory. |

### Sample code

```
package org.techhub;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
public class ClientApplication {
    public static void main(String[] args) {
        Resource r=new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
    }
}
```

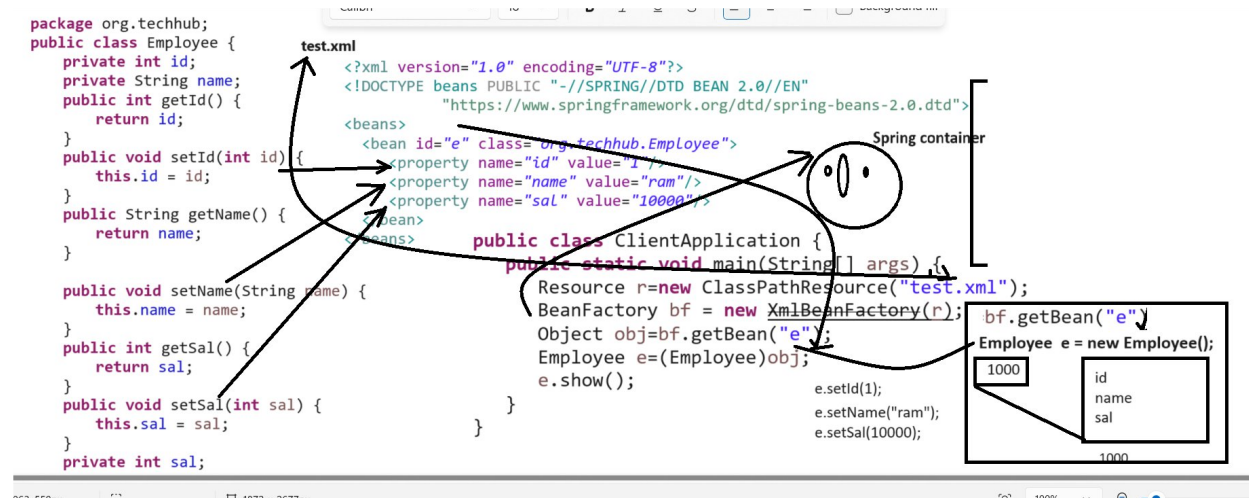
### 3. Call getBean() method of spring container for perform dependency injection

getBean() method is member of BeanFactory which can accept the id of bean and create its object at run time and call setter method automatically means perform setter injection by default

**Syntax:** Object getBean("bean id from xml");

### Sample code

```
package org.techhub;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
public class ClientApplication {
    public static void main(String[] args) {
        Resource r=new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        Object obj=bf.getBean("e");
        Employee e=(Employee)obj;
        e.show();
    }
}
```



### Description:

If we think about above diagram we have main method class and in main we have statement **Resource r = new ClassPathResource("test.xml");** this statement indicate we load the xml file in application where we configure our bean classes and these mark with bean classes mark as resource here Resource r indicate the reference of Resource interface and new ClassPathResource is implementer class of Resource interface and we create its object for load xml file from class path

Then again we have statement `BeanFactory bf = new XMLBeanFactory(Resource)` here we create spring container using `BeanFactory` interface

Again we have statement `Object obj = bf.getBean("e")` this statement indicate we access e bean id from xml file and spring container internally create object of `Employee` class using that id like as `Employee e = new Employee()` and this process known as inversion of control after creating object of `Employee` class spring container internally call `setter()` methods like as `e.setId(1)`; `e.setName("ram")`; `e.setSal(10000)` this process known as dependency injection or setter injection but setter injection is type of dependency injection

**Example:** WAP to perform setter injection using spring core?

---

We want to create class name as `Product` with id, name and price and perform setter injection on it.

**POJO.java**

```
package org.techhub;

public class Product {
    private int id;
    private String name;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }

    public int getPrice() {
        return price;
    }
    public void setPrice(int price) {
        this.price = price;
    }
    public String toString() {
        return "["+id+", "+name+", "+price+"]";
    }
    private int price;
}
```

```

}
product.xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">

<beans>
  <bean id="p" class="org.techhub.Product">
    <property name="id" value="1"/>
    <property name="name" value="Parle-G"/>
    <property name="price" value="5"/>
  </bean>
</beans>

```

### **ProductClientApplication.java**

```

package org.techhub;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

public class ProductClientApplication {
    public static void main(String[] args) {
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        Product p=(Product)bf.getBean("p");
        /*
        * internal code of getBean()
        * Product p = new Product();
        * p.setId(1);
        * p.setName("Parle-G");
        * p.setPrice(5);
        * setter injection dependency
        */
        System.out.println(p);
    }
}

```

### **Constructor Injection**

---



Constructor injection means constructor call by spring container as well parameter pass to constructor by spring container called as constructor injection

**Note:** when we have default constructor in class then we not required or not need to write logic of constructor injection to perform constructor injection and automatically perform by spring container

### Example of default constructor injection

The diagram illustrates the process of default constructor injection. It consists of three parts: a Java class, an XML configuration file, and a main application class. The Java class `Test` has a default constructor. The XML file `test.xml` defines a bean `t` of type `org.techhub.consinject.Test`. The main application class `ConstructorInjectionApplication` uses `XmlBeanFactory` to load the XML file and retrieve the bean `t` using the name `"t"`. A callout box shows the equivalent manual instantiation: `Test t = new Test();`. Arrows indicate the flow of information: from the XML file to the `main` method, and from the `Test` class to the `main` method.

```
1 package org.techhub.consinject;
2
3 public class Test {
4     public Test() {
5         System.out.println("I am default constructor");
6     }
7 }
8 test.xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">
<beans>
  <bean id="t" class="org.techhub.consinject.Test">
  </bean>
</beans>
9 public class ConstructorInjectionApplication {
10     public static void main(String[] args) {
11         Resource r = new ClassPathResource("test.xml");
12         BeanFactory bf = new XmlBeanFactory(r);
13         Test t=(Test)bf.getBean("t");
14     }
15 }
```

If we think about above diagram we have one class name as `Test` with default constructor and we have one more class with main method in main method we have statement `Test t=(Test)bf.getBean("t")` this statement indicate spring container create object of `Test` class like as `Test t = new Test()` means by default constructor get executed automatically when spring container create object of the class.

### Complete source code with example

#### test.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">
<beans>
  <bean id="p" class="org.techhub.Product">
    <property name="id" value="1"/>
    <property name="name" value="Parle-G"/>
    <property name="price" value="5"/>
  </bean>
  <bean id="t" class="org.techhub.consinject.Test">
  </bean>
</beans>
```

#### Client Application

```
package org.techhub.consinject;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.context.annotation.ClassPathBeanDefinitionScanner;
```

```

import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
public class ConstructorInjectionApplication {
    public static void main(String[] args) {
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        Test t=(Test)bf.getBean("t");
    }
}

```

**Note:** when we have parameterized constructor in class then we need to perform constructor injection manually means need to write logic for pass parameter to constructor bean using xml

### How to pass parameter to constructor by using bean

```

<bean id="idname" class="classname">
    <constructor-arg value="value" index="parameter index" type="datatype"/>
</bean>

```

**constructor-arg:** If we want to think about above bean tag we have constructor-arg this tag indicate we want to pass parameter to constructor

**value:** value attribute indicate parameter value of constructor

**index:** index attribute indicate parameter number and the index start from 0<sup>th</sup> location

**type:** type indicate data type of parameter.

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">
<beans>
    <bean id="t" class="org.techhub.consinject.Test">
        <constructor-arg value="Ramesh" type="java.Lang.String" index="0"></constructor-arg>
        <constructor-arg value="1" type="int" index="1"/>
        <constructor-arg value="10000" type="int" index="2"/>
    </bean>
</beans>

```

```

Test.java
package org.techhub.consinject;
public class Test {
    public Test(String name,int id,int sal)
    {
        System.out.println("Name is "+name);
        System.out.println("Id is "+id);
        System.out.println("Salary is "+sal);
    }
}

```

```

public class ConstructorInjectionApplication {
    public static void main(String[] args) {
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        Test t=(Test)bf.getBean("t");
    }
}

```

Test t = new Test("Ramesh",1,10000);  
 Note: create object by spring contain and pass parameter to constructor from object

If we think about above code we have statement `Test t=(Test) bf.getBean("t")` the meaning of this statement is spring container create internally object of Test class and pass three parameter to it.  
`Test t = new Test("Ramesh",1,10000);`

### Complete source code of above diagram

```

package org.techhub.consinject;
public class Test {

```

```

        public Test(String name, int id, int sal) {
            System.out.println("Name is "+name);
            System.out.println("Id is "+id);
            System.out.println("Salary is "+sal);
        }
    }
}

```

#### test.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">

<beans>
    <bean id="p" class="org.techhub.Product">
        <property name="id" value="1"/>
        <property name="name" value="Parle-G"/>
        <property name="price" value="5"/>
    </bean>
    <bean id="t" class="org.techhub.consinject.Test">
        <constructor-arg value="Ramesh" type="java.lang.String" index="0"></constructor-arg>
        <constructor-arg value="1" type="int" index="1"/>
        <constructor-arg value="10000" type="int" index="2"/>
    </bean>
</beans>

```

#### ClientApplication.java

```

package org.techhub.consinject;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.context.annotation.ClassPathBeanDefinitionScanner;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

public class ConstructorInjectionApplication {
    public static void main(String[] args) {
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        Test t=(Test)bf.getBean("t");
    }
}

```

#### Collection dependency

---

Collection dependency means when we pass collection as parameter to class function or constructor called as collection dependency

When we pass collection as parameter to setter method called as collection with setter injection and when we pass collection as parameter to constructor called as collection with constructor injection

## How to manage the collection dependency using XML

```
<bean id="idname" class="classname">
  <property name="propertyname">
    <collectiontype>
      <value>data</value>
    </collectiontype>
  </property>
</bean>
```

### Note: collection dependency with constructor

```
<constructor-arg>
  <list>
    <value>Gurugram</value>
    <value>Haryana</value>
    <value>India</value>
  </list>
</constructor-arg>
```

**<property name="propertyname">**: this statement indicate setter method name

**<collectiontype>**: Here we need to specify type of collection which we use as parameter in method

**<value>**: this tag decide actual value stored in collection

```
class Employee
{ private List<String> list;
  public void setNames(List<String> list)
  { this.list=list;
  }
  public void show()
  { for(String name:list)
    { System.out.println(name);
    }
  }
}
```

```
<bean id="e" class="org.techhub.colldep.Employee">
  <property name="names">
    <list>
      <value>Ram</value>
      <value>Shyam</value>
      <value>Ganesh</value>
    </list>
  </property>
</bean>
```

ClientApplication.

```
Resource r=new ClassPathResource("test.xml");
BeanFactory bf=new XmlBeanFactory(r);
Employee emp=(Employee)bf.getBean("e");
emp.show();
```

```
Employee e = new Employee();
List<String>list=new ArrayList<String>();
list.add("Ram");
list.add("Shyam");
list.add("Ganesh");
e.setNames(list);
```



Spring container

if we think about code we have statement in class Employee public void setNames(List<String> list) this statement indicate we have collection dependency using list collection so we use the list tag in xml file under names property and if we think about main method we have Statement Employee emp=(Employee)bf.getBean("e") here spring container create object of Employee class like as Employee

e= new Employee() and create reference of List collection like as List<String> list=new ArrayList<String>() and add data in mention in xml in list collection like as list.add("Ram"); list.add("Shyam"); list.add("Ganesh") and pass the list collection as parameter to setNames() function like as emp.setNames(list) and when we call emp.show() then all list data get displayed.

#### **test.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">

<beans>
  <bean id="e" class="org.techhub.colldep.Employee">
    <property name="names">
      <list>
        <value>Ram</value>
        <value>Shyam</value>
        <value>Ganesh</value>
      </list>
    </property>
  </bean>
</beans>
```

#### **Employee.java**

```
package org.techhub.colldep;
import java.util.*;
public class Employee {
    List<String> list;
    public void setNames(List<String> list) {
        this.list=list;
    }
    public void show() {
        for(String name:list) {
            System.out.println(name);
        }
    }
}
```

#### **CollDepApplication.java**

```
package org.techhub.colldep;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
```

```

public class CollDepApplication {
    public static void main(String[] args) {
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        Employee emp=(Employee)bf.getBean("e");
        emp.show();
    }
}

```

## Map Dependency

---

Map dependency means if we pass Map parameter to setter function or to constructor called as Map dependency.

When we pass Map to the setter method then it is called as map with setter dependency injection

When we pass map to the constructor then it is called as map with constructor dependency injection

### How to manage the Map dependency by using XML configuration

---

If we want to manage the map dependency by using XML configuration we have to use following syntax.

```

<bean id="b" class="classname">
    <property name="propertyname">
        <map>
            <entry key="key" value="value"/>
        </map>
    </propertyname>
</bean>

```

### Example:

#### Company.java

```

package org.techhub.mapdep;
import java.util.*;
public class Company {
    private Map map;
    public void setNames(Map<Integer, String> map) {
        this.map=map;
    }
    public void show() {
        Set<Map.Entry<Integer, String>> set=map.entrySet();
        for(Map.Entry<Integer, String> e:set) {
            System.out.println(e.getKey()+"\t"+ e.getValue());
        }
    }
}

```

### test.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">

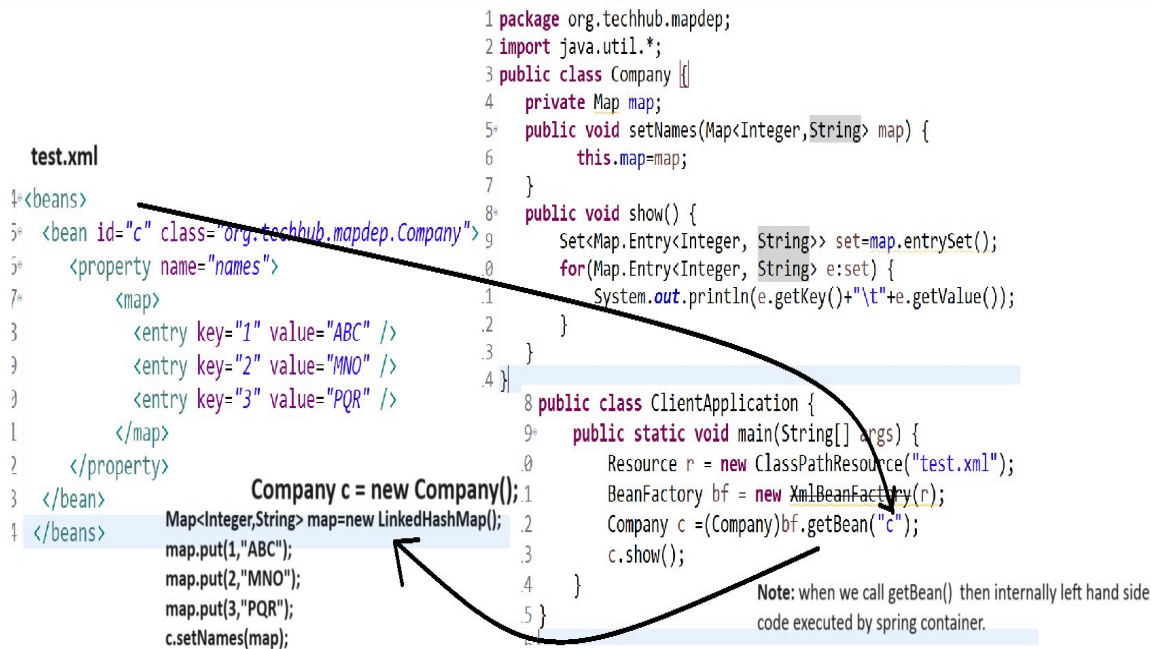
<beans>
  <bean id="c" class="org.techhub.mapdep.Company">
    <property name="names">
      <map>
        <entry key="1" value="ABC" />
        <entry key="2" value="MNO" />
        <entry key="3" value="PQR" />
      </map>
    </property>
  </bean>
</beans>
```

### ClientApplication.java

```
package org.techhub.mapdep;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

public class ClientApplication {
    public static void main(String[] args) {
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        Company c =(Company)bf.getBean("c");
        c.show();
    }
}
```

Following Diagram shows the meaning of above code.



### Note : map with constructor dependency injection

```
<bean id="company" class="com.geeksforgeeks.org.Company">
  <constructor-arg>
    <map>
      <entry key="101" value="Software development"></entry>
      <entry key="102" value="Software testing"></entry>
      <entry key="103" value="Security"></entry>
    </map>
  </constructor-arg>
</bean>
```

## Object dependency

Object dependency means if we pass object as parameter in function or in constructor called as object dependency.

When we pass object as parameter to setter method then it is called as object with setter dependency injection

When we pass object as parameter to constructor then it is called as object with constructor dependency injection.

**Example:** Following diagram denote object dependency without spring framework using plain core java



```
class Employee
{
    private int id;
    private String name;
    private int sal;

    //setter and getters
}
```

```
class Company
{
    private Employee employee;
    public void setEmployee(Employee employee)
    { this.employee=employee;
    }
    public void showEmployee()
    {
        System.out.println(employee.getId()+"\t"+employee.getName()+"\t"+employee.getSal());
    }
}

Company c = new Company();
Employee e1 =new Employee();
e1.setId(1);
e1.setName("ABC");
e1.setSal(10000);
c.setEmployee(e1);
```

**Note:** If we think about left hand side we have Company class and in Company class we have method name as void setEmployee(Employee employee) this method cannot call without employee object means we can say setEmployee() is dependent on Employee object and setEmployee() is Company class so indirectly we can say Company is also dependent on Employee class object means we can say Employee and Company maintain the object dependency in terms of spring

**Note:** if we think about diagram we have class name as Company which is dependent on Employee object and we pass this object in setEmployee() so we need to create manually object of Employee class and pass to setEmployee() function but if we manage this object dependency by spring framework then we not need to create object of employee class it is responsibility of spring framework to create object of employee class or parameter class just we specify configuration within XML file

## How to configure object dependency in spring framework

If we want to configure object dependency in spring framework we have to use following Syntax

```
<bean id="parameter bean " class="parameter class name">
    <property name="name of property" value="value"/>
    <property name="name of property" value=""/>
</bean>

<bean id="dependent id" class="dependent classname">
    <property name="propertyname" ref="parameter bean"/>
</bean>
```

### Example:

#### Employee.java

```
package org.techhub.objdep;

public class Employee {
    private int id;
    private String name;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
```

```

        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getSal() {
        return sal;
    }
    public void setSal(int sal) {
        this.sal = sal;
    }
    private int sal;
}

```

#### Company.java

```

package org.techhub.objdep;

public class Company {
    private Employee employee;

    public void setEmployee(Employee employee) {
        this.employee=employee;
    }
    public void show() {
        System.out.println(employee.getId()+"\t"+employee.getName()+"\t"+employee.getSal());
    }
}

```

#### ObjDepApplication.java

```

package org.techhub.objdep;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
public class ObjDepClientApplication {
    public static void main(String[] args) {
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        Company c=(Company)bf.getBean("c");
        c.show();
    }
}

```

Following diagram demonstrate the working of object dependency



## Auto wiring

Auto wiring concept only works with object dependency if we use the auto wiring then we not need to call manually reference of parameter bean from dependent class

**Autowiring** in the Spring framework can inject dependencies automatically. The Spring container detects those dependencies specified in the configuration file and the relationship between the beans. This is referred to as **Autowiring in Spring**. To enable Autowiring in the Spring application we should use `@Autowired` annotation. Autowiring in Spring internally uses constructor injection. An autowired application requires fewer lines of code comparatively but at the same time, it provides very little flexibility to the programmer.

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">

<beans>
    <bean id="e" class="org.techhub.objdep.Employee">
        <property name="id" value="1"/>
        <property name="name" value="ABC"/>
        <property name="sal" value="10000"/>
    </bean>
    <bean id="c" class="org.techhub.objdep.Company">
        <property name="employee" ref="e"/>
    </bean>
</beans>

```

**Note:** if we think about above XML configuration we have two beans one is Employee and one is Company and Company is dependent bean and Employee is parameter bean so we call Employee bean id manually from Company bean by using ref attribute of property tag  
So if we use the auto wiring concept we not required to call parameter bean manually from dependent bean

### Types of auto wiring concept

---

- 1. byName:** byName auto wiring means you have to provide reference declared within dependent class and within bean id must be same
- 2. byType:** byType auto wiring means match data type mention within dependent class and bean configured property must be same
- 3. byConstructor :** if we want to manage object dependency by using constructor we can use byConstructor auto wiring
- 4. auto-detect:** auto-detect help us to manage the setter injection object dependency as well as constructor with object dependency using byType
- 5. default:** it is used for default auto wiring technique of spring framework and it is byType auto wiring

### Modes of Autowiring

| Modes       | Description                                                                                                      |
|-------------|------------------------------------------------------------------------------------------------------------------|
| No          | This mode tells the framework that autowiring is not supposed to be done. It is the default mode used by Spring. |
| byName      | It uses the name of the bean for injecting dependencies.                                                         |
| byType      | It injects the dependency according to the type of bean.                                                         |
| Constructor | It injects the required dependencies by invoking the constructor.                                                |
| Autodetect  | The autodetect mode uses two other modes for autowiring – constructor and byType.                                |

### How to use auto wiring practically

---

**If we want to use auto wiring practically we have to use following syntax**

**Syntax:**

```
<bean id="ref" class="dependent classname" autowire="autowiring type">  
  <property name="propertyname" value="value"/>  
</bean>
```

</bean>

```
1 package org.techhub.objdep;
2
3 public class Company {
4     private Employee employee;
5
6     public void setEmployee(Employee employee) {
7         this.employee=employee;
8     }
9     public void show() {
10        System.out.println(employee.getId()+"\t"+employee.getName()+"\t"+employee.getSal());
11    }
12 }
13
```

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
3     "https://www.springframework.org/dtd/spring-beans-2.0.dtd">
4 <beans>
5     <bean id="e" class="org.techhub.objdep.Employee">
6         <property name="id" value="1"/>
7         <property name="name" value="ABC"/>
8         <property name="sal" value="10000"/>
9     </bean>
10    <bean id="c" class="org.techhub.objdep.Company" autowire="byName">
11    </bean>
12 </beans>
```

Exception in thread "main" java.lang.NullPointerException: Cannot invoke "org.techhub.objdep.  
at org.techhub.objdep.Company.show(Company.java:10)  
at org.techhub.objdep.ObjDepClientApplication.main(ObjDepClientApplication.java:12)

**Note:** if we think about above diagram we get NullPointerException because in configure we use the byName auto wiring and we give different name of Employee bean in XML and different name of reference of Employee in Company class so we get error because byName auto wiring indicate we must be give bean id and reference of parameter bean class in dependent class must be same.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
3     "https://www.springframework.org/dtd/spring-beans-2.0.dtd">
4 <beans>
5     <bean id="employee" class="org.techhub.objdep.Employee">
6         <property name="id" value="1"/>
7         <property name="name" value="ABC"/>
8         <property name="sal" value="10000"/>
9     </bean>
10    <bean id="c" class="org.techhub.objdep.Company" autowire="byName">
11    </bean>
12 </beans>
```

Must be same in byName  
auto wiring concept

```
1 package org.techhub.objdep;
2 public class Company {
3     private Employee employee;
4     public void setEmployee(Employee employee) {
5         this.employee=employee;
6     }
7     public void show() {
8         System.out.println(employee.getId()+"\t"+employee.getName()+"\t"+employee.getSal());
9     }
10 }
```

**Complete source code of above example**

**Employee.java**

```
package org.techhub.objdep;
```

```
public class Employee {
    private int id;
    private String name;
    public int getId() {
        return id;
    }
}
```

```

public void setId(int id) {
    this.id = id;
}
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public int getSal() {
    return sal;
}
public void setSal(int sal) {
    this.sal = sal;
}
private int sal;
}

```

#### Company.java

```

package org.techhub.objdep;
public class Company {
    private Employee employee;
    public void setEmployee(Employee employee) {
        this.employee=employee;
    }
    public void show() {
        System.out.println(employee.getId()+"\t"+employee.getName()+"\t"+employee.getSal());
    }
}

```

#### ObjDepClientApplication.java

```

package org.techhub.objdep;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
public class ObjDepClientApplication {
    public static void main(String[] args) {
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        Company c=(Company)bf.getBean("c");
    }
}

```

```

        c.show();
    }
}

```

#### test.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">

<beans>
    <bean id="employee" class="org.techhub.objdep.Employee">
        <property name="id" value="1"/>
        <property name="name" value="ABC"/>
        <property name="sal" value="10000"/>
    </bean>
    <bean id="c" class="org.techhub.objdep.Company" autowire="byName">
    </bean>
</beans>

```

#### byType auto wiring concept

---

#### test.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">

<beans>
    <bean id="employee" class="org.techhub.objdep.Employee">
        <property name="id" value="1"/>
        <property name="name" value="ABC"/>
        <property name="sal" value="10000"/>
    </bean>
    <bean id="c" class="org.techhub.objdep.Company" autowire="byType">
    </bean>
</beans>

```

#### Employee.java

```

package org.techhub.objdep;

public class Employee {
    private int id;
    private String name;
    public int getId() {
        return id;
    }
}

```

```

public void setId(int id) {
    this.id = id;
}
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public int getSal() {
    return sal;
}
public void setSal(int sal) {
    this.sal = sal;
}
private int sal;
}

```

#### **Company.java**

```

package org.techhub.objdep;
public class Company {
    private Employee employee;
    public void setEmployee(Employee employee) {
        this.employee=employee;
    }
    public void show() {
        System.out.println(employee.getId()+"\t"+employee.getName()+"\t"+employee.getSal());
    }
}

```

#### **ObjClientApplication.java**

```

package org.techhub.objdep;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
public class ObjDepClientApplication {
    public static void main(String[] args) {
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        Company c=(Company)bf.getBean("c");
        c.show();
    }
}

```



```
}
```

### By constructor auto wiring

**When we use auto wiring using constructor then we need to write constructor in dependent class**

---

#### test.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">

<beans>
    <bean id="employee" class="org.techhub.objdep.Employee">
        <property name="id" value="1"/>
        <property name="name" value="ABC"/>
        <property name="sal" value="10000"/>
    </bean>
    <bean id="c" class="org.techhub.objdep.Company" autowire="constructor">
    </bean>
</beans>
```

#### Employee.java

```
package org.techhub.objdep;

public class Employee {
    private int id;
    private String name;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getSal() {
        return sal;
    }
    public void setSal(int sal) {
        this.sal = sal;
    }
}
```

```
private int sal;  
}
```

#### Company.java

```
package org.techhub.objdep;  
public class Company {  
    private Employee employee;  
  
    public Company(Employee employee) {  
        this.employee=employee;  
    }  
  
    public void show() {  
        System.out.println(employee.getId()+"\t"+employee.getName()+"\t"+employee.getSal());  
    }  
}
```

#### ObjClientApplication.java

```
package org.techhub.objdep;  
  
import org.springframework.beans.factory.BeanFactory;  
import org.springframework.beans.factory.xml.XmlBeanFactory;  
import org.springframework.core.io.ClassPathResource;  
import org.springframework.core.io.Resource;  
public class ObjDepClientApplication {  
    public static void main(String[] args) {  
        Resource r = new ClassPathResource("test.xml");  
        BeanFactory bf = new XmlBeanFactory(r);  
        Company c=(Company)bf.getBean("c");  
        c.show();  
    }  
}
```

#### autodetect

```
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"  
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">  
<beans>  
    <bean id="employee" class="org.techhub.objdep.Employee">  
        <property name="id" value="1"/>  
        <property name="name" value="ABC"/>  
        <property name="sal" value="10000"/>  
    </bean>
```

```
<bean id="c" class="org.techhub.objdep.Company" autowire="autodetect">
</bean>
</beans>
```

#### **Employee.java**

```
package org.techhub.objdep;
```

```
public class Employee {
    private int id;
    private String name;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getSal() {
        return sal;
    }
    public void setSal(int sal) {
        this.sal = sal;
    }
    private int sal;
}
```

#### **Company.java**

```
package org.techhub.objdep;
```

```
public class Company {
    private Employee employee;

    public Company(Employee employee) {
        this.employee=employee;
    }

    public void show() {
        System.out.println(employee.getId()+"\t"+employee.getName()+"\t"+employee.getSal());
    }
}
```

### ObjDepClientApplication.java

```
package org.techhub.objdep;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
public class ObjDepClientApplication {
    public static void main(String[] args) {
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        Company c=(Company)bf.getBean("c");
        c.show();
    }
}
```

### How to configure spring application by using annotations

---

#### Q. What is annotation?

---

Java Annotation is a tag that represents the *metadata* i.e. attached with class, interface, methods or fields to indicate some additional information which can be used by java compiler and JVM.

Basically Annotation is inbuilt interfaces and mark with @ symbol as well as user can create own annotation and annotation help us to configure the java application

#### Q. What is benefit of annotation in spring framework?

---

The major goal of annotation is avoid XML configurations mean you can write all configuration of spring without XML by using annotations.

#### If we want to work with spring framework we need to know the some important annotations

1. **@Configuration:** this annotation help us to write the configuration class where we configure bean means it work like as XML file where we configured bean files
2. **@Bean:** this annotation helps us to configure bean means using this annotation we create instance of bean class.
3. **@ComponentScan:** this annotation helps us to configure or scan the packages path like as service packages, repository packages etc.
4. **@Value:** this annotation helps us to perform setter injection or configure the properties with XML file.
5. **@Required:** this annotation is used for apply compulsion to pass value to setter method
6. **@Autowired:** this annotation is used for perform auto wiring with object dependency

7. **@Qualifier:** this annotation help us to decide which child class object should create with parent reference means using this annotation we can achieve dynamic polymorphism by using spring framework.
8. **@Scope:** this annotation decides the scope of the bean means singleton, proto type, session, request, application etc.
9. **@Service:** this annotation helps us to create the service layer
10. **@Repository:** this annotation helps us to create the repository layer
11. **@Controller:** this annotation is used for create the controller in spring MVC etc.

## How to configure spring core Application by using annotations

---

### 1. Create Maven Project

### 2. Add following maven dependency

```
<dependencies>
  <!-- https://mvnrepository.com/artifact/org.springframework/spring-core -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-core</artifactId>
    <version>5.3.19</version>
  </dependency>
  <!-- https://mvnrepository.com/artifact/org.springframework/spring-context -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.3.19</version>
  </dependency>
</dependencies>
```

### 3. Write POJO class as Bean class

```
package org.techhub;

import org.springframework.beans.factory.annotation.Value;

public class Employee {
    @Value("1")
    private int id;
    @Value("ganesh")
    private String name;

    public int getId() {
        return id;
    }

    public void setId(int id) {
```

```

        this.id = id;
    }

    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getSal() {
        return sal;
    }
    public void setSal(int sal) {
        this.sal = sal;
    }
    }
    @Value("10000")
    private int sal;
}

```

#### 4. Writer configuration class and configure bean using annotations

---

**Configuration class work as XML file where we configure all bean classes**

If we want to use configuration class we have @Configuration annotation and for bean configuration purpose we have @Bean annotation.

```

package org.techhub;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
@Configuration
public class Config
{
    @Bean(name="e")
    public Employee getEmployee() {
        return new Employee();
    }
}

```

#### 5. Write Client Application and call bean

---

```

package org.techhub;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
public class ClientAnnotationApplication {

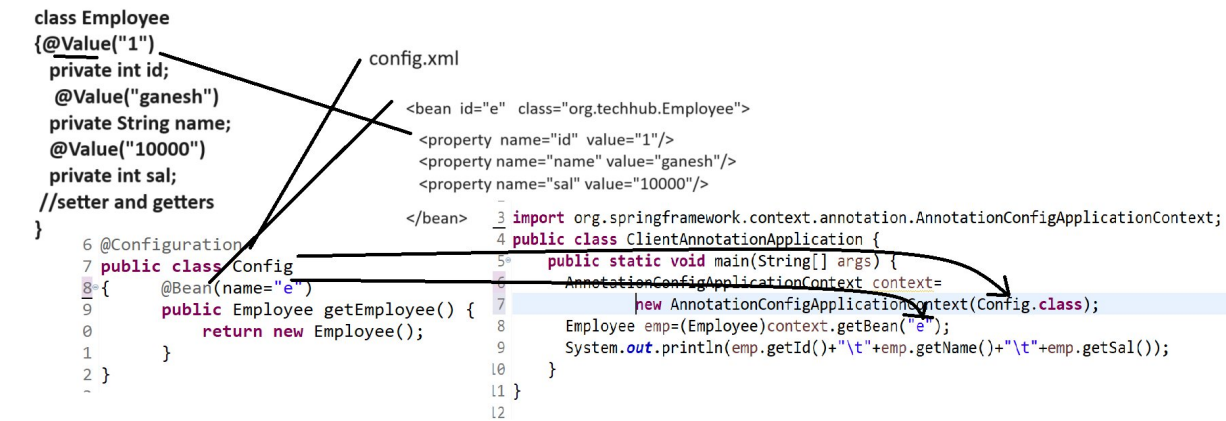
```

```

public static void main(String[] args) {
    AnnotationConfigApplicationContext context=
        new AnnotationConfigApplicationContext(Config.class);
    Employee emp=(Employee)context.getBean("e");
    System.out.println(emp.getId()+"\t"+emp.getName()+"\t"+emp.getSal());
}
}

```

**Example:** following diagram shows how to work application by using annotations



## How to perform object dependency by using annotation

if we want to object dependency by using annotation we have can use `@Autowired` annotations

### Employee.java

```

package org.techhub;
import org.springframework.beans.factory.annotation.Value;
public class Employee {
    @Value("1")
    private int id;
    @Value("ganesh")
    private String name;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
}

```

```

        public void setName(String name) {
            this.name = name;
        }
        public int getSal() {
            return sal;
        }
        public void setSal(int sal) {
            this.sal = sal;
        }
        @Value("10000")
        private int sal;
    }

```

#### **Company.java**

```

package org.techhub;
import org.springframework.beans.factory.annotation.Autowired;
public class Company {
    @Autowired//Employee employee=new Employee();
    private Employee employee;
    public void setEmployee(Employee employee) {
        this.employee=employee;
    }
    public void show() {
        System.out.println(employee.getId()+"\t"+employee.getName()+"\t"+employee.getSal());
    }
}

```

#### **Config.java**

```

package org.techhub;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
@Configuration
public class Config
{
    @Bean(name="e")
    public Employee getEmployee() {
        return new Employee();
    }
    @Bean(name="c")
    public Company getCompany() {
        return new Company();
    }
}

```

#### **ClientApplication.java**

```

package org.techhub;

```



```

import org.springframework.context.annotation.AnnotationConfigApplicationContext;
public class ClientAnnotationApplication {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context=
            new AnnotationConfigApplicationContext(Config.class);
        Company c=(Company)context.getBean("c");
        c.show();
    }
}

```

**@Qualifier annotation: it is used for to work with dynamic polymorphism**

**Example without using spring core and @Qualifier annotation**

```

package org.techhub.usingcore;
interface Vehicle{
    void engine();
}
class Bike implements Vehicle{
    public void engine() {
        System.out.println("I am bike");
    }
}
class Car implements Vehicle{
    public void engine() {
        System.out.println("I am car");
    }
}
class ShowRoom{
    public void setVehicle(Vehicle vehicle) {
        vehicle.engine();
    }
}
public class VehicleApplication {
    public static void main(String[] args) {
        ShowRoom sr = new ShowRoom();
        Vehicle v= new Bike();
        sr.setVehicle(v);
        v=new Car();
        sr.setVehicle(v);
    }
}

```

**Note:** above code indicate implementation of dynamic polymorphism using plain core java and we want To achieve this task by using spring core then we can use the @Autowired and @Qualifier annotations.

if want to achieve dynamic polymorphism using spring framework then use the interface reference or parent class reference with @Autowired annotation and use the bean of child in @Qualifier annotation whose object want to create and store its address in parent reference shown in following code.

### **Example using @Autowired and @Qualifier**

#### **Vehicle.java**

```
package org.techhub.dynamicpoly;
public interface Vehicle {
    void engine();
}
```

#### **Car.java**

```
package org.techhub.dynamicpoly;
public class Car implements Vehicle {
    public void engine() {
        System.out.println("I am Car");
    }
}
```

#### **Config.java**

```
package org.techhub.dynamicpoly;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
@Configuration
public class Config {
    @Bean(name="bike")
    public Bike getBike() {
        return new Bike();
    }
    @Bean(name="car")
    public Car getCar() {
        return new Car();
    }
    @Bean(name="s")
    public ShowRoom getShowRoom() {
        return new ShowRoom();
    }
}
```

#### **ShowRoom.java**

```
package org.techhub.dynamicpoly;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
public class ShowRoom {
```

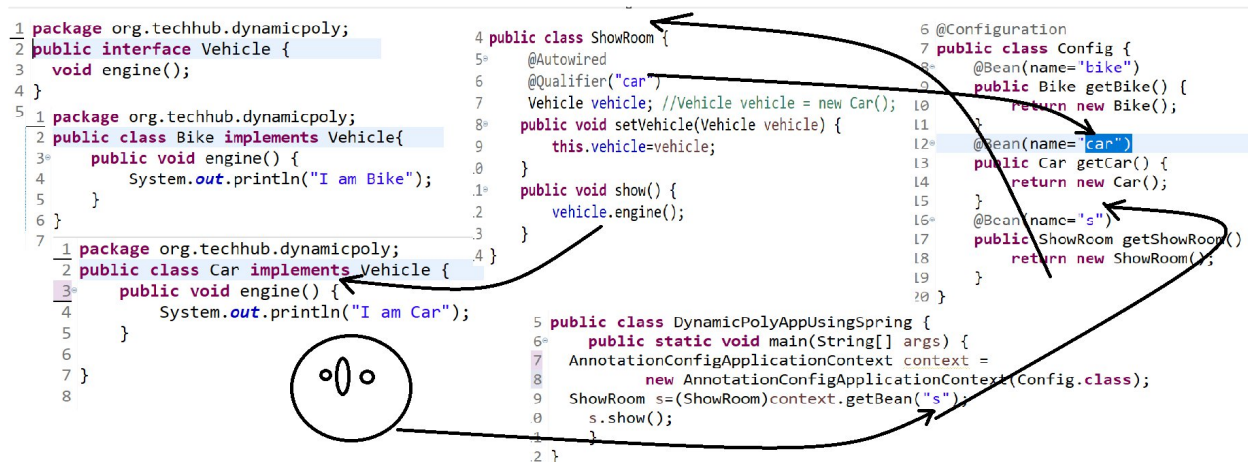
```

    @Autowired
    @Qualifier("car")
    Vehicle vehicle; //Vehicle vehicle = new Car();
    public void setVehicle(Vehicle vehicle) {
        this.vehicle=vehicle;
    }
    public void show() {
        vehicle.engine();
    }
}
}

ClientApplication.java
package org.techhub.dynamicpoly;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
public class DynamicPolyAppUsingSpring {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context =
            new AnnotationConfigApplicationContext(Config.class);
        ShowRoom s=(ShowRoom)context.getBean("s");
        s.show();
    }
}

```

Following diagram shows the working of @Autowired and @Qualifier annotation



## Spring Bean Scope

Bean Scope decide how we can create bean object and life of bean object called as bean scope

**Bean Scopes** refers to the lifecycle of Bean that means when the object of Bean will be instantiated, how long does that object live, and how many objects will be created for that bean throughout. Basically, it controls the instance creation of the bean and it is managed by the spring container.

### Types of bean scope

1. **singleton:** singleton scope is a default scope of spring bean and it is used for create only one object of bean class in whole application.

2. **prototype:** proto type scope means bean can create new object every time when we call `getBean()` method.

**Note:** remaining scope we will discuss in spring MVC

3. **session:**

4. **application**

5. **page**

6. **request**

Etc.

1. **Singleton:** Only one instance will be created for a single bean definition per Spring IoC container and the same object will be shared for each request made for that bean.
2. **Prototype:** A new instance will be created for a single bean definition every time a request is made for that bean.
3. **Request:** A new instance will be created for a single bean definition every time an HTTP request is made for that bean. But only valid in the context of a web-aware Spring ApplicationContext.
4. **Session:** Scopes a single bean definition to the lifecycle of an HTTP Session. But only valid in the context of a web-aware Spring ApplicationContext.
5. **Global-Session:** Scopes a single bean definition to the lifecycle of a global HTTP Session. It is also only valid in the context of a web-aware Spring ApplicationContext.

### How to set the spring bean scope by using XML and by using Annotations

---

#### How to set the bean scope using XML

---

##### Syntax:

```
<bean id="idname" class="classname" scope="scopename">  
</bean>
```

##### Example using singleton scope

---

##### test.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"  
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">  
<beans>  
    <bean id="t" class="org.techhub.scope.Test" scope="singleton">
```

```
</bean>
</beans>
```

```
package org.techhub.scope;

public class Test {

    public Test() {
        System.out.println("I am constructor");
    }
}
```

### ClientApplication.java

```
package org.techhub.scope;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.context.ApplicationContext;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
public class ScopeClientApplication {
    public static void main(String[] args) {
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf =new XmlBeanFactory(r);
        Test t1=(Test)bf.getBean("t");
        Test t2=(Test)bf.getBean("t");
        Test t3=(Test)bf.getBean("t");
        Test t4=(Test)bf.getBean("t");

    }
}
```

### Output:

```
I am constructor
```

**Note:** if we think about above code we have call getBean () method 4 times but constructor get executed only once means our Test class created single object in whole application.

### Example using proto type bean

---

#### test.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">

<beans>
```

```
<bean id="t" class="org.techhub.scope.Test" scope="prototype">
</bean>
</beans>
```

### Test.java

```
package org.techhub.scope;
public class Test {

    public Test() {
        System.out.println("I am constructor");
    }
}
```

### ClientApplication.java

```
package org.techhub.scope;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.context.ApplicationContext;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
public class ScopeClientApplication {
    public static void main(String[] args) {
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf =new XmlBeanFactory(r);
        Test t1=(Test)bf.getBean("t");
        Test t2=(Test)bf.getBean("t");
        Test t3=(Test)bf.getBean("t");
        Test t4=(Test)bf.getBean("t");

    }
}
```

### Output

```
<terminated> ScopeClientApplication [Java Application] C:\Users\Admin\p2\pool\p
I am constructor
I am constructor
I am constructor
I am constructor
```

**Note:** if we think about above we call `getBean()` method 4 times so our constructor get executed 4 times because we have scope prototype and prototype scope create new object every time when we call `getBean()` method.

### How to set the bean scope by using annotations

---

#### Test.java

```
package org.techhub.scope;

public class Test {
    public Test() {
        System.out.println("I am constructor");
    }
}
```

#### Config.java

```
package org.techhub.scope;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Scope;
```

@Configuration

```
public class Config {
    @Bean(name="t")
    @Scope(value = "singleton")
    public Test getTest() {
        return new Test();
    }
}
```

#### ScopeClientApplication.java

```
package org.techhub.scope;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

public class ScopeClientApplication {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context= new
        AnnotationConfigApplicationContext(Config.class);
        Test t1=(Test)context.getBean("t");
    }
}
```

```

        Test t2=(Test)context.getBean("t");
        Test t3=(Test)context.getBean("t");
        Test t4=(Test)context.getBean("t");

    }
}

```

### Example of prototype by using annotation

#### Test.java

```

package org.techhub.scope;

public class Test {
    public Test() {
        System.out.println("I am constructor");
    }
}

```

#### Config.java

```

package org.techhub.scope;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Scope;

```

@Configuration

```

public class Config {
    @Bean(name="t")
    @Scope(value = "prototype")
    public Test getTest() {
        return new Test();
    }
}

```

#### ScopeClientApplication.java

```

package org.techhub.scope;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

public class ScopeClientApplication {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context= new
        AnnotationConfigApplicationContext(Config.class);
    }
}

```



```

    Test t1=(Test)context.getBean("t");
    Test t2=(Test)context.getBean("t");
    Test t3=(Test)context.getBean("t");
    Test t4=(Test)context.getBean("t");

}
}

```

## Spring bean life cycle

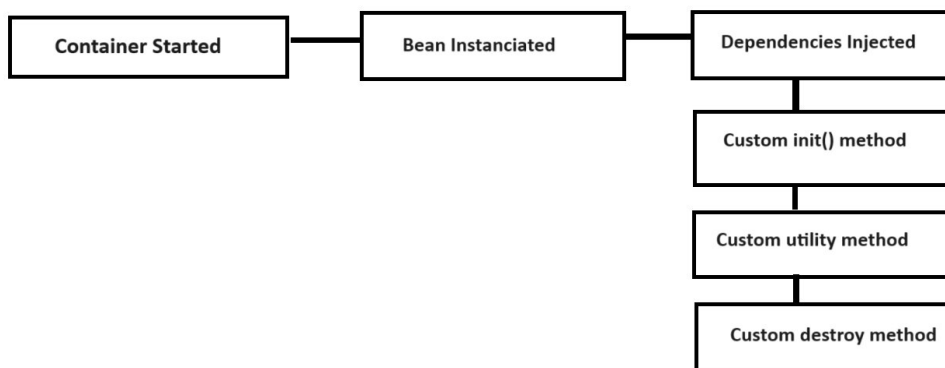
Spring bean life cycle means how spring bean object born or created by spring container and how it behave throughout its life and when and how its dies.

If we think about spring bean life cycle then first container get started after that the container create instance of bean as per the request or scope and perform dependency injection and after that call utility methods and close the bean object

The bean life cycle refers to when & how the bean is instantiated, what action it performs until it lives, and when & how it is destroyed.

Bean life cycle is managed by the spring container. When we run the program then, first of all, the spring container gets started. After that, the container creates the instance of a bean as per the request, and then dependencies are injected. And finally, the bean is destroyed when the spring container is closed. Therefore, if we want to execute some code on the bean instantiation and just after closing the spring container, then we can write that code inside the custom **init()** method and the **destroy()** method.

How spring bean life cycle work shown in following diagram



How to implement spring bean life cycle practically

If we want to implement the spring bean life cycle practically we have three approaches.

1. By XML Configuration
2. By using Annotations
3. By using Programming approach

## How to implement the spring bean life cycle by using XML configuration

---

### 1. Create Bean class and write custom init and custom destroy in it.

```
package org.techhub.scope;

public class Employee {
    private int id;
    private String name;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getSal() {
        return sal;
    }

    public void setSal(int sal) {
        this.sal = sal;
    }

    private int sal;

    public void userInit() {
        System.out.println("custom init method");
    }
    public void userDestroy() {
        System.out.println("Custom destroy method");
    }
    public void show(){
        System.out.println("Custom utility method");
    }
}
```

## 2. Configure bean in XML and configure init and destroy method with bean

### Syntax:

```
<bean id="idname" class="classname" init-method="init method name" destroy-  
method="methodname">  
</bean>
```

### Example with source code

#### test.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"  
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">  
  
<beans>  
    <bean id="t" class="org.techhub.scope.Test" scope="prototype">  
</bean>  
    <bean id="e" class="org.techhub.lifecycle.Employee" init-method="userInit" destroy-  
method="userDestroy">  
        <property name="id" value="1"/>  
        <property name="name" value="ABC"/>  
        <property name="sal" value="10000"/>  
</bean>  
</beans>
```

#### Employee.java

```
package org.techhub.lifecycle;
```

```
public class Employee {  
    private int id;  
    private String name;  
    public Employee() {  
        System.out.println("I am constructor class");  
    }  
    public int getId() {  
        return id;  
    }  
  
    public void setId(int id) {  
        this.id = id;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

```

    }

    public void setName(String name) {
        System.out.println("setter method executed");
        this.name = name;
    }

    public int getSal() {
        return sal;
    }

    public void setSal(int sal) {
        this.sal = sal;
    }

    private int sal;

    public void userInit() {
        System.out.println("init method");
    }

    public void userDestroy() {
        System.out.println("destroy method");
    }

    public void show() {
        System.out.println("user custom method");
    }
}

```

#### ClientApplication.java

```

package org.techhub.lifecycle;
import org.springframework.context.ConfigurableApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class ClientApplication {
    public static void main(String[] args) {
        ConfigurableApplicationContext ap = new ClassPathXmlApplicationContext("test.xml");
        Employee emp=(Employee) ap.getBean("e");
        emp.show();
        ap.close();
    }
}

```

## Bean Life Cycle Implementation by using annotations

---

If we want to implement the bean life cycle by using annotation we required to use two annotations with init method and with destroy method.

**@PostConstruct:** this annotation helps us to call the init method means if we use @PostConstruct annotation with method then method work as init method

**@PreDestroy :** this annotation help us to call the destroy method and if we use the @PreDestroy annotation with method then method work as destroy method

**Note:** When we use the @PostConstruct and @PreDestroy annotation we need to use the following maven dependency

`<!-- https://mvnrepository.com/artifact/javax.annotation/javax.annotation-api -->`

```
<dependency>
  <groupId>javax.annotation</groupId>
  <artifactId>javax.annotation-api</artifactId>
  <version>1.3.2</version>
</dependency>
```

## Complete Example with source code

---

```
package org.techhub.lifecycle;
import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;
import org.springframework.beans.factory.annotation.Value;
public class Employee {
```

```
    @Value("1")
    private int id;
    @Value("ABC")
    private String name;

    public Employee() {
        System.out.println("I am constructor class");
    }
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }
}
```

```

    public String getName() {
        return name;
    }

    public void setName(String name) {

        this.name = name;
    }

    public int getSal() {
        return sal;
    }

    public void setSal(int sal) {
        this.sal = sal;
    }
    @Value("10000")
    private int sal;

    @PostConstruct
    public void userInit() {
        System.out.println("init method");
    }
    @PreDestroy
    public void userDestroy() {
        System.out.println("destroy method");
    }
    public void show() {
        System.out.println("user custom method");
    }
}

Config.java
package org.techhub.lifecycle;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class Config {

    @Bean(name="emp")
    public Employee getEmployee() {

```

```

        return new Employee();
    }
}

```

### ClientApplication

```

package org.techhub.lifecycle;
import org.springframework.context.ConfigurableApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class ClientApplication {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context= new
AnnotationConfigApplicationContext(Config.class);
        Employee emp=(Employee) context.getBean("emp");
        emp.show();
        context.close();
    }
}

```

## 3. Programming Approach

---

when we want to implement spring bean life cycle by using programming approach we not need to configure XML or not need to use the annotations just we required implement two interfaces name as **InitializingBean** and **DisposableBean**

If we think about **InitializingBean** interface then this interface provide **afterPropertiesSet ()** method to us which work as init method just we need to override it.

If we think about **DisposableBean** interface then this interface provide **destroy ()** method to us which work as destroy method just we need to override it.

### Example with complete source code

#### Employee.java

```

package org.techhub.lifecycle;

import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;

import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;
import org.springframework.beans.factory.annotation.Value;

public class Employee implements InitializingBean, DisposableBean {

```

```

    @Value("1")
    private int id;
    @Value("ABC")
    private String name;

    public Employee() {
        System.out.println("I am constructor class");
    }
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {

        this.name = name;
    }

    public int getSal() {
        return sal;
    }

    public void setSal(int sal) {
        this.sal = sal;
    }
    @Value("10000")
    private int sal;
    public void show() {
        System.out.println("user custom method");
    }
    public void destroy() throws Exception {

        System.out.println("I am work as destroy method ");
    }
}

```



```

public void afterPropertiesSet() throws Exception {
    System.out.println("I am work as init method");
}
}

```

#### Config.java

```

package org.techhub.lifecycle;

```

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

```

```

@Configuration

```

```

public class Config {

    @Bean(name="emp")
    public Employee getEmployee() {
        return new Employee();
    }
}

```

#### ClientApplication.java

```

package org.techhub.lifecycle;
import org.springframework.context.ConfigurableApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

```

```

public class ClientApplication {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context= new
AnnotationConfigApplicationContext(Config.class);
        Employee emp=(Employee) context.getBean("emp");
        emp.show();
        context.close();
    }
}

```

#### Difference between component and componentScan annotations

Using the annotation @ComponentScan , you can tell Spring where do your Spring-managed components lie. These Spring-Managed components could be annotated with @Repository, @Service, @Controller and ofcourse @Component.

For example - Lets say your spring-managed components lie inside 2 packages com.example.test1 and com.example.test2. Then your componentScan would be something like this

```
@ComponentScan(basePackages="com.example.test1","com.example.test2")
```

On the other hand, @Component is a generic annotation for any Spring-Managed component. For example - If you create a class called Testing inside the package com.example.test1 and annotate with Spring @Component.

```
@Component
Class Testing
{
    public Testing()
    {
    }

    public void doSomething()
    {
        System.out.println("do something");
    }
}
```

### Difference between bean and autowired

#### @Bean

- **Purpose:** The @Bean annotation is used at the method level in a @Configuration-annotated class to define a bean.
- **Function:** It tells Spring that the method will return an object that should be registered as a bean in the Spring application context.
- **Usage:** Typically used when you want to manually configure and instantiate a bean. For example, you might want to customize the creation of an object, set properties, or inject other dependencies.

#### @Autowired

- **Purpose:** The @Autowired annotation is used at the field, constructor, or method level to automatically inject a dependency.
- **Function:** It tells spring to resolve and inject the specified dependency automatically. It is a way to achieve automatic wiring of dependencies, leveraging Spring's IoC (Inversion of Control) container.
- **Usage:** Commonly used to automatically inject dependencies without needing explicit configuration. For example, you might want to inject a service or repository into another class.

In summary, `@Bean` is for defining and registering beans within the Spring context, while `@Autowired` is for injecting those beans into other components.

### Difference between `@Component` and `@Bean` annotation

#### `@Component` Annotation:

- **Purpose:** It is a class-level annotation that tells Spring to automatically detect and manage the annotated class as a Spring bean.
- **Usage:** This is used mainly in component scanning. When you annotate a class with `@Component` (or other derived stereotypes like `@Service`, `@Repository`, or `@Controller`), Spring will automatically register that class as a bean, provided that the class is in a package that is scanned by Spring (via `@ComponentScan` or automatic scanning of the root package).
- **Automatic Scanning:** Classes marked with `@Component` or its specializations are picked up through classpath scanning.

```
@Component
public class MyService {
    // service logic
}
```

- **Specialized Annotations:**

- `@Service`: for service-layer components.
- `@Repository`: for persistence-layer components.
- `@Controller`: for controller components in MVC architecture.

#### 2. `@Bean` Annotation:

- **Purpose:** It is a method-level annotation used in a `@Configuration` class. It explicitly declares a single bean, and you provide the configuration of that bean within the method.
- **Usage:** This is used when you need to manually define a bean in a Java configuration class rather than relying on Spring's component scanning. It provides more control over the creation of beans, such as initializing third-party classes or when specific configurations are required.

```
@Configuration
public class MyConfig {

    @Bean
    public MyService myService() {
        return new MyService();
    }
}
```

- **Explicit Declaration:** Beans are explicitly defined using `@Bean` within configuration classes.

## Key Differences:

| Aspect                        | @Component                                          | @Bean                                            |
|-------------------------------|-----------------------------------------------------|--------------------------------------------------|
| Annotation Location           | Class level                                         | Method level in @Configuration classes           |
| Bean Creation                 | Automatically registered through scanning           | Explicitly registered by declaring the method    |
| Use Case                      | When class is annotated for component scanning      | When manual bean creation is required            |
| Control over Initialization   | Less control over bean creation                     | Full control over bean initialization            |
| Usage with External Libraries | Not applicable for third-party classes or libraries | Used to register third-party or external classes |

## When to Use:

- **@Component:** When you're working with your own classes, and automatic component scanning is sufficient for your use case.
- **@Bean:** When you need to register third-party classes, need more control over bean instantiation, or need a more explicit bean definition.